

T^{0 9} fixtures^{1 5} -- flake^{0 0 7}
quarantine^{0 2 2} ledger (plan T-
C ; DEC- ; FR- ,
FR-)

Fixtures for `test_flake_ledger_red.sh` , proving `gates/flake_ledger.py` (not yet implemented) correctly identifies a flaky gate per DEC-15: "a gate whose verdict differs across ≥ 2 of its last 10 runs on an unchanged cache key is flaky."

Scenarios

Dir	verdicts (oldest- first)	minor- ity	expected
<code>fl_good_flagged_within_10/</code>	P F P F P F P F P F	5	FLAKY, quarantined, cache-excluded
<code>fl_negc- trl_determinist- ic_never_flagged/</code>	P P P P P P P P P P	0	STABLE, cache-eligible
<code>fl_bound- ary_one_dissent_stable/</code>	P P P P P P P P P F	1	STABLE (below the ≥ 2 threshold)

`_shared/gate_flaky_50_50.sh` -- a real toy gate driven by a counter file so the RED test can reproduce a SPECIFIC 10-run PASS/FAIL sequence deterministically (never real randomness -- §11.4.50).

`_shared/gate_deterministic.sh` -- a real toy gate that always PASSES on its (unchanged) input; the negative control's live-invocation counterpart.

`_shared/inputs/fixed_input.txt` -- the unchanged input driving `gate_deterministic.sh` across all 10 runs.

0 5 7

What this RED test does NOT cover (honest scope, per T /T /T)

precedent -- owed to T-C09's implementer, not silently tested here)

- The **owner + deadline** fields of a quarantine entry: DEC-15/§11.4.248 require them, but neither names a concrete owner-assignment or deadline-computation rule, so this RED test only asserts a quarantine entry EXISTS with non-empty owner/deadline fields, never their specific values.
- The **monotone-decreasing ratchet** property across MULTIPLE separate test runs (only a single-run snapshot is exercised here) -- a ratchet is inherently a property of repeated invocations over time, which is T-C10's (backstop lane) territory per plan.md, not this RED test's fixture set.
- **non-blocking-in-fast-lane vs blocking-at-release-seam**: this RED test exercises `flake_ledger.py`'s own `check / cache-eligible` subcommands directly; it does NOT exercise the fast-lane vs release-seam INTEGRATION (that lives in whichever T-C0x tool consumes `flake_ledger.py`'s verdict at each seam -- out of scope for a unit-level RED test on the ledger itself).